

Loading CLIP with PyTorch

<https://gemini.google.com/app/133acc1f5dd0d86b>

User prompt: how to load clip with pytorch

Response: You can load a CLIP model in PyTorch using a few different libraries depending on your needs. The three most common approaches use **OpenAI's official repository**, **Hugging Face Transformers**, or **OpenCLIP**.

Option 1: Using OpenAI's Original CLIP Library

This is the standard approach if you want to use the exact weights and architecture released by OpenAI.

1. Installation:

```
pip install torch torchvision
pip install git+https://github.com/openai/CLIP.git
```

2. Loading Code:

```
import torch
import clip

# Set device
device = "cuda" if torch.cuda.is_available() else "cpu"

# Load the model and image preprocessing pipeline
model, preprocess = clip.load("ViT-B/32", device=device)

model.eval() # Set to evaluation mode
print(f"Model loaded successfully on {device}")
```

Option 2: Using Hugging Face Transformers (transformers)

If you are already using the Hugging Face ecosystem or want cleaner integration with tokenizers and config files, use CLIPModel.

1. Installation:

```
pip install torch transformers
```

2. Loading Code:

```
import torch
from transformers import CLIPModel, CLIPProcessor

device = "cuda" if torch.cuda.is_available() else "cpu"

# Load model and processor (handles both tokenization and image preprocessing)
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32").to(device)
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

model.eval()
```

Option 3: Using OpenCLIP (open_clip_torch)

If you need newer, larger, or community-trained variants (like LAION models, ViT-H/14, ViT-g/14, etc.), OpenCLIP is the preferred package.

1. Installation:

```
pip install torch open_clip_torch
```

2. Loading Code:

```
import torch
import open_clip

device = "cuda" if torch.cuda.is_available() else "cpu"

# Create model, preprocessing pipeline, and tokenizer
model, _, preprocess = open_clip.create_model_and_transforms(
    'ViT-B-32', pretrained='laion2b_s34b_b79k'
)
tokenizer = open_clip.get_tokenizer('ViT-B-32')

model = model.to(device)
model.eval()
```

Sources

1. [CLIP/README.md at main · openai/CLIP - GitHub](#)

User prompt: what about sentence_transformers

Response: Using **sentence-transformers** is a great choice if you want a high-level, streamlined API to encode both text and images into the *same* shared vector space. It handles tokenization, image transformations, and device placement automatically under a clean `model.encode()` syntax.

1. Installation

```
pip install -U sentence-transformers
```

2. Loading and Using a CLIP Model

You can load pre-integrated CLIP models directly using SentenceTransformer:

```
import torch
from sentence_transformers import SentenceTransformer
from PIL import Image
import requests

# Set device
device = "cuda" if torch.cuda.is_available() else "cpu"

# Load a CLIP model wrapper (supports both text and images)
model = SentenceTransformer('clip-ViT-B-32', device=device)

# --- Example: Encoding Text and Images ---

# 1. Encode text queries
text_embeddings = model.encode([
    "A dog playing in the snow",
    "A red sports car"
])
```

```
# 2. Encode images (pass PIL Image objects, URLs, or local file paths)
image_url = "https://unsplash.com/photos/QtXgNsmJQSs/download?w=640"
image = Image.open(requests.get(image_url, stream=True).raw)

image_embeddings = model.encode(image)

# 3. Compute similarity (cosine similarity)
similarity = model.similarity(text_embeddings, image_embeddings)
print("Similarity scores:", similarity)
```

Why use sentence-transformers for CLIP?

- **Unified API:** You use the exact same `.encode()` function for both images and text.
- **Multilingual Support:** It hosts cross-lingual variants (like `clip-ViT-B-32-multilingual-v1`) that let you search for images using 50+ different languages out-of-the-box.
- **Convenience:** It abstracts away separate processors, tokenizers, and manual tensor conversions.

Sources

1. [sentence-transformers/clip-ViT-B-32-multilingual-v1 - ModelScope](#)
-